Architettura degli Elaboratori

Gestione delle eccezioni

Davide Polese

davide.polese@uniroma1.it

Special thanks and credits:

Alessandro Checco, Andrea Sterbini, Iacopo Masi, Claudio di Ciccio

[S&PdC] 4.9, 5.8

Esercizio

Nota: Assumiamo che
`la s0, vettore`
sia un'istruzione del set standard. In realtà, è una pseudoistruzione

Esercizio (esame 16-9-14)

1. Individuare i **data** e **control hazard**
   - assumendo la **decisione** dei **branch** in fase **ID**
2. Individuare i casi in cui il **forwarding è applicabile**
3. Individuare gli **stalli necessari**
4. Calcolare la **durata totale** in cicli di clock **con forwarding**
5. Calcolare la **durata senza forwarding**
6. **Riordinare le istruzioni** per ridurre gli stalli (con fwd.)
7. Calcolare la **durata** del programma **ottimizzato** (**4.**)

```
.data
vettore: .word 7, 14, … , 48, 91 # 70 pari, 30 dispari
.text
main:
        addi t0, x0, 0              # somma parziale
        la   s0, vettore            # ind. base
        addi  t1, s0 , 396          # offset ultima word
ciclo:
        lw   t3, 0(t1)
        andi t2, t3, 1          # è dispari?
        beq t2, x0, salta       # se non lo è, salto
        add  t0, t0, t3         # altrimenti, lo sommo
salta:
        addi t1, t1, -4         # prossimo elemento
        bge  t1, s0, ciclo      # fine del ciclo?
        addi a7, x0, 1          # stampa…
        mv   a0, t0             # … la somma
        ecall
```

Hazard, stalli, durata con forwarding (punti 1., 2., 3., 4.)

```
.data
vettore: .word 7, 14, … , 48, 91 # Tot. 70 pari, 30 dispari
.text
main:
      addi t0, x0, 0          F D E M W @5°
      la   s0, vettore          F D E M W @6°
      addi  t1, s0 , 396          F D E M W @7°
ciclo:
      lw   t3, 0(t1)        #     F D E M W @8°      » F D E M W @18°
      andi t2, t3, 1        #       » F D E M W @10°     » F D E M W @20°
      beq t2, x0, salta     #           » F D E M W @12°     » F D E M W @22°
      add  t0, t0, t3       #               F D E M W @13°
salta:
      addi t1, t1, -4       #                 F D E M W @14°    » F D E M W @24°
      bge  t1, s0, ciclo    #                   » F D E M W @16°    » F D E M W @26°
      addi a7, x0, 1        #                                                    @1007°
      mv   a0, t0           # ciclo = 30*(6+3+1) + 70*(5+3+2) - 1                @1008°
      ecall                 # totale = 4 + 3 + 999 + 3 = 1009                    @1009°
```

4 (riempim.)

control hazard
(1 stallo) tranne in entrata del ciclo

control hazard
(1 stallo)

10 colpi di clock (dispari)

10 colpi di clock (pari)

Hazard, stalli, durata senza forwarding (punto 5.)

```
.data
vettore: .word 7, 14, … , 48, 91 # Tot. 70 pari, 30 dispari
.text
main:
    addi t0, x0, 0          F D E M W @5°
    la   s0, vettore          F D E M W @6°
    addi  t1, s0 , 396          » » F D E M W @9°
ciclo:
    lw   t3, 0(t1)            #     » » F D E M W @12°          » F D E M W @25°
    andi t2, t3, 1            #           » » F D E M W @15°        » » F D E M W @28°
    beq t2, x0, salta         #                 » » F D E M W @18°        …        @31°
    add  t0, t0, t3           #                       F D E M W @19°
salta:
    addi t1, t1, -4           #                         F D E M W @20°           @33°
    bge  t1, s0, ciclo        #                           » » F D E M W @23°     @36°
    addi a7, x0, 1            #                                                  @1311°
    mv   a0, t0               # ciclo = 30*(6+6+1) + 70*(5+6+2) - 1              @1312°
    ecall                     # totale = 4 + 3 + 2 + 2 + 1299 + 3 = 1313         @1313°
```

4 (riempim.)

13 colpi di clock (dispari)

13 colpi di clock(pari)

solo in entrata nel ciclo

Riordinamento con forwarding (punto 6.)

```
.data
vettore: .word 7, 14, … , 48, 91 # Tot. 70 pari, 30 dispari
.text
main:
      addi t0, x0, 0              F D E M W @5°
      la   s0, vettore              F D E M W @6°
      addi  t1, s0 , 396              F D E M W @7°
ciclo:
      lw   t3, 0(t1)              #     F D E M W @8°      » F D E M W @18°
      andi t2, t3, 1              #       » F D E M W @10°     » F D E M W @20°
      beq t2, x0, salta           #           » F D E M W @12°    » F D E M W @22°
      add  t0, t0, t3             #                F D E M W @13°
salta:
      addi t1, t1, -4             #                  F D E M W @14°    » F D E M W @24°
      bge  t1, s0, ciclo          #                    » F D E M W @16°   » F D E M W @26°
      addi a7, x0, 1              #                                                        @1007°
      mv   a0, t0                 # ciclo = 30*(6+3+1) + 70*(5+3+2) - 1                    @1008°
      ecall                       # totale = 4 + 3 + 999 + 3 = 1009                        @1009°
```

Riordinamento con forwarding (punto 6.)

2 stalli sono stati **eliminati** all'interno di un **ciclo ripetuto 100** volte
→ **200 cicli di clock** necessari **in meno**
→ **risparmio** sul totale di **~20%**

```
.data
vettore: .word 7, 14, … , 48, 91 # Tot. 70 pari, 30 dispari
.text
main:
      addi t0, x0, 0              F D E M W @5°
      la   s0, vettore              F D E M W @6°
      addi  t1, s0 , 396              F D E M W @7°
ciclo:                                          8 colpi di clock (dispari)
      lw   t3, 0(t1)              #     F D E M W @8°      » F D E M W @16°
      andi t2, t3, 1              #       » F D E M W @10°     » F D E M W @18°
      addi t1, t1, -4             #           F D E M W @11°       F D E M W @19°
      beq t2, x0, salta           #             F D E M W @12°
      add  t0, t0, t3
salta:                            #               F D E M W @13°     » F D E M W @21°
      bge  t1, s0, ciclo          #                 F D E M W @14°       F D E M W @22°
      addi a7, x0, 1              #                                                    @807°
      mv   a0, t0                 # ciclo = 30*(6+1+1) + 70*(5+1+2) - 1                @808°
      ecall                       # totale = 4 + 3 + 799 + 3 = 809                     @809°
```

Non rimovibile mediante rioardinam.

8 colpi di clock (pari)

Cosa c'è in pipeline a t=16? (punto 7.) (con forwarding)

```
.data
vettore: .word 7, 14, … , 48, 91 # Tot. 70 pari, 30 dispari
.text
main:
    addi t0, x0, 0          F D E M W @5°
    la   s0, vettore          F D E M W @6°
    addi  t1, s0 , 396          F D E M W @7°
ciclo:
    lw   t3, 0(t1)          #     F D E M W @8      » F D E M W @18°
    andi t2, t3, 1          #       » F D E M W @10     » F D E M W @20°
    beq t2, x0, salta       #           » F D E M W @12     » F D E M W @22°
    add  t0, t0, t3         #               F D E M W @13°
salta:
    addi t1, t1, -4         #                 F D E M W @14°    » F D E M W @24°
    bge  t1, s0, ciclo      #                   » F D E M W @16°    » F D E M W @26°
    addi a7, x0, 1          #                                                   @1007°
    mv   a0, t0             # ciclo = 30*(6+3+1) + 70*(5+3+2) - 1               @1008°
    ecall                   # totale = 4 + 3 + 999 + 3 = 1009                   @1009°
```

Eccezioni

SAPIENZA
UNIVERSITÀ DI ROMA

Immagine: https://sonic.fandom.com/wiki/Time_Warp_Plate

Eccezioni ed interrupt

Eventi imprevisti
Eccezione: qualsiasi cambiamento non previsto dal flusso di controllo
Interrupt: evento con cause esterne

Tipo di evento	Provenienza	Terminologia RISC-V
Richiesta di I/O	Esterna	Interrupt
ecall	Interna	Eccezione
Errore ALU / FPU (overflow, div. per 0, ...)	Interna	Eccezione
Utilizzo di un'istruzione non definita*	Interna	Eccezione
Malfunzionamenti hardware	Entrambi	Eccezione o Interrupt
Page fault	Interna	Eccezione
Accesso ad indirizzo di mem. inesistente	Interna	Eccezione

* Può essere gestita a livello software (ma il compilatore può non conoscere l'architettura esatta)

Gestire l'eccezione

Gestione:

- se si tratta di una **ecall** → **rispondere all'evento**
- se si tratta di un **errore** risolvibile → **riparare la situazione**

Hardware

1. Bloccare l'istruzione corrente
2. Completare le istruzioni precedenti l'eccezione (se possibile)
3. Svuotare la pipeline dalle istruzioni successive all'eccezione
4. Scrivere la causa dell'eccezione
5. Salvare l'indirizzo dell'istruzione responsabile
6. Trasferire il controllo all'indirizzo pre-assegnato

Sistema Operativo

1. Esaminare la causa dell'eccezione
2. Agire di conseguenza
   - Ad es., richiesta di eseguire un'istruzione non implementata → kill
   - Ad es., page fault → caricare in memoria una pagina dal disco
3. Restituire il controllo al processo (se possibile)

Gestire l'eccezione

Gestione:

- se si tratta di una **ecall** → **rispondere all'evento**
- se si tratta di un **errore** risolvibile → **riparare la situazione**

Primo step: preservare lo stato della computazione

- **Salvataggio del PC** in un registro apposito (**SEPC**) per poter ripartire (se possibile)
- **Salvataggio dello stato** (come una chiamata a subroutine)

Occorre agire **come un control-hazard → flush**

- dell'istruzione che ha generato l'**eccezione**/ricevuto l'**interruzione**
- **delle istruzioni seguenti** in pipeline

- Overflow o errore (EXE) → le tre istruzioni in IF, ID ed EXE
- Interruzione esterna (ID) → l'istruzione in IF
- ecall (EXE) → le tre istruzioni in IF, ID ed EXE
- Istruzione non definita (ID) → l'istruzione in IF

Gestire l'eccezione

Per poter rispondere all'evento occorre **individuarne la causa**

- Si usa un registro dedicato (registro **SCAUSE**)
  - **Polled exception handling**
  - **Controlla tutti i dispositivi associati all'exception code associato all'eccezione asserita**
- In alternativa si usano **interruzioni vettorizzate**
  - indirizzo del codice di gestione della risposta direttamente determinato dal tipo di eccezione
  - **Vector interrupt table (contiene l'indirizzo iniziale delle Interrupt Service Routine)**
  - **Vector interrupt controller (mappa l'interrupt all'elemento della vector interrupt table)**

Secondo step: eseguire la routine di gestione dell'interruzione

1. Cambiare PC
   - occorre aggiungere al MUX del PC un ingresso con un indirizzo prefissato
     - 0x1C090000
     - Supervisor Trap Vector (STVEC)
2. Tornare ad **eseguire il codice iniziale** (se possibile)
   - ripristinare **PC ⟵ SEPC** e lo stato

Interrupt vettorizzati (PIC)

PIC24 Devices

Interrupt Vector Table

Reset – GOTO Instruction	000000
Reset – GOTO Address	000002
Oscillator Fail Trap Vector	000004
Address Error Trap Vector	
Stack Error Trap Vector	
Math Error Trap Vector	
. . .	
Interrupt Vector 0	000014
Interrupt Vector 1	000016
. . .	
Interrupt Vector 52	00007C
Interrupt Vector 53	00007E
Interrupt Vector 54	000080
. . .	
Interrupt Vector 116	0000FC
Interrupt Vector 117	0000FE

dsPIC33 Devices

Reset – GOTO Instruction	000000
Reset – GOTO Address	000002
Oscillator Fail Trap Vector	000004
Address Error Trap Vector	
Hard Trap Vector	
Stack Error Trap Vector	
Math Error Trap Vector	
. . .	
Interrupt Vector 0	000014
Interrupt Vector 1	000016
. . .	
Interrupt Vector 116	0000FC
Interrupt Vector 117	0000FE
Interrupt Vector 118	000100
. . .	
Interrupt Vector 244	0001FC
Interrupt Vector 245	0001FE

- Ogni vector contiene l'indirizzo di partenza della Interrupt Service Routine (ISR) con l'istruzione di salto
  Può essere eseguita una sola istruzione!

Interrupt vettorizzati (ARM)

Exception number	IRQ number	Vector	Offset
495	479	IRQ479	0x7BC
. . .		. . .	. . .
18	2	IRQ2	0x48
17	1	IRQ1	0x44
16	0	IRQ0	0x40
15	-1	SysTick	0x3C
14	-2	PendSV	0x38
13		Reserved	0x30
12	-4	DebugMonitor	
11	-5	SVCall	0x2C
10		Reserved	
9			
8			
7			
6	-10	UsageFault	0x18
5	-11	BusFaults	0x14
4	-12	MemManage	0x10
3	-13	HardFault	0x0C
2	-14	NMI	0x08
1		Reset	0x04
		Initial SP value	0x00

Cortex-M33

- Ogni vector ha la dimensione di una parola,
- Può essere eseguita una sola istruzione!
- Il registro VTOR contiene l'indirizzo base della tabella.

Interrupt RISC-V

h a r d w a r e

- **scause.interrupt** = 1; **scause.exception_code** = **I**
- **sstatus.spie** = 1, save previous interrupt enable. (See ISR stack.)
- **sstatus.spp** = Previous privilege mode (s or u).
- **sstatus.sie** = 0, interrupts are disabled unless the ISR re-writes this.
- **sepc** = Interrupted **PC**, save the return address.
- IF **stvec.mode** == Direct THEN; **PC** = (**stvec.Base** & ~0xF)
- IF **stvec.mode** == Vectored THEN; **PC** = (**stvec.Base** & ~0xF) + (**I** * 4)

The exit procedure for a supervisor interrupt when sret is executed.

- **sstatus.mie** = **sstatus.mpie**, restore interrupt enable
- Privilege mode = **sstatus.spp**, restore privilege (supervisor=1, user=0)
- **sstatus.spie** = 1
- **sstatus.spp** = 0, Set to 0 when SRET is executed.
- **PC** = **sepc**

SCAUSE register

SXLEN-1	SXLEN-2 … 0
Interrupt	Exception Code (WLRL)
1	SXLEN-1

Figure 52. Supervisor Cause register `scause`.

Interrupt	Exception Code	Description
1	0	*Reserved*
1	1	Supervisor software interrupt
1	2-4	*Reserved*
1	5	Supervisor timer interrupt
1	6-8	*Reserved*
1	9	Supervisor external interrupt
1	10-12	*Reserved*
1	13	Counter-overflow interrupt
1	14-15	*Reserved*
1	≥16	*Designated for platform use*

Interrupt	Exception Code	Description
0	0	Instruction address misaligned
0	1	Instruction access fault
0	2	Illegal instruction
0	3	Breakpoint
0	4	Load address misaligned
0	5	Load access fault
0	6	Store/AMO address misaligned
0	7	Store/AMO access fault
0	8	Environment call from U-mode
0	9	Environment call from S-mode
0	10-11	*Reserved*
0	12	Instruction page fault
0	13	Load page fault
0	14	*Reserved*
0	15	Store/AMO page fault
0	16-17	*Reserved*
0	18	Software check
0	19	Hardware error
0	20-23	*Reserved*
0	24-31	*Designated for custom use*
	32-47	*Reserved*
	48-63	*Designated for custom use*
	≥64	*Reserved*

Gestire l'eccezione: il registro Status (in RISC-V)

31	30 … 20	19	18	17
SD	WPRI	MXR	SUM	WPRI
1	11	1	1	1

16 … 15	14 … 13	12 … 11	10 … 9	8	7	6	5	4 … 2	1	0
XS[1:0]	FS[1:0]	WPRI	VS[1:0]	SPP	WPRI	UBE	SPIE	WPRI	SIE	WPRI
2	2	2	2	1	1	1	1	3	1	1

Figure 42. Supervisor-mode status register (`sstatus`) when SXLEN=32.

SIE Interrupt enable: Viene posto ad 1 se e solo se l'interrupt è abilitato
SPIE Previous IE: contiene il valore del SIE durante la gestione dell'eccezione
SPP privilege level: Viene posto a 0 se il processore è in user mode (1 altrimenti)

$12 (status)	12	0x0000ff11

$12 (status)	12	0x0000ff13

Gestire l'eccezione: nuove istruzioni e priorità

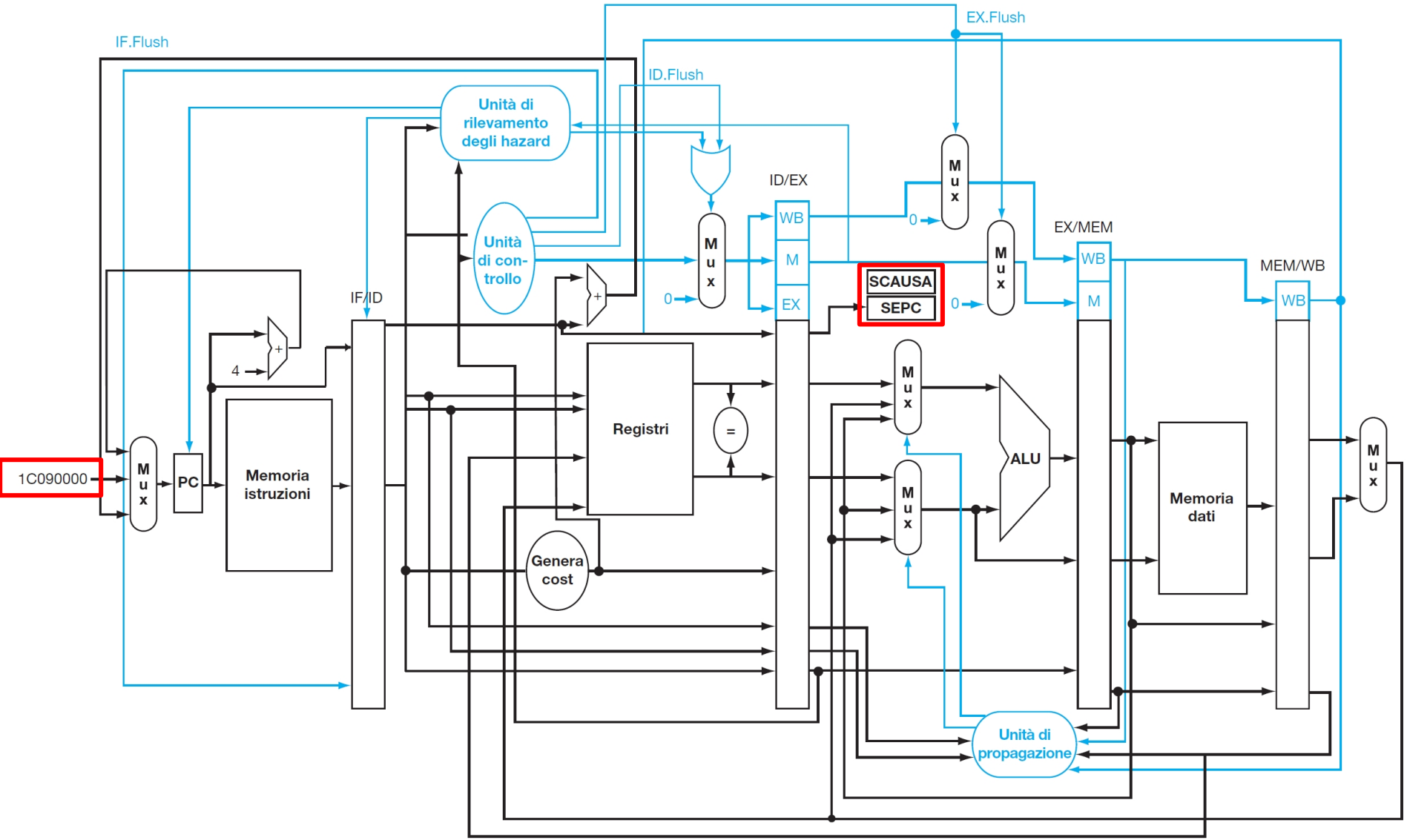
Nuove istruzioni necessarie

- `sret` (return from exception in machine or supervisor mode)
- `csrw csr, rs1` (write crs register)
- `csrr rd, csr` (read crs register)

Se **più interruzioni** avvengono **contemporaneamente**:

- L'hardware dà una **priorità** alle interruzioni più importanti
  - Priorità più alta a interrupt più **time-critical**
  - Nel registro **scause** sono indicate tutte le interruzioni attive
  - Gestioni possibili:
    1. Disattivare ulteriori interruzioni
       - Perdita di informazioni possibile
    2. Interrompere la routine di gestione per altre interruzioni a priorità più alta
       - Parte della gestione da rifare

Gestione delle eccezioni



Esercizio per casa: da vj a vjal

Si vuole aggiungere alla CPU l'istruzione vectorised jump (**vj**), di tipo **I** e sintassi assembly

```
vj indice, x0, vettore
```

che salta all'indirizzo contenuto nell'elemento indice-esimo del *vettore di word* . Il secondo registro (rs2) non è utilizzato e per questo viene settato a zero

L'assemblatore inserirà nella parte immediata, i 12 bit di scostamento relativi al program counter per raggiungere l'indirizzo di memoria dell'etichetta *vettore*. In altre parole, l'indirizzo di memoria dell'etichetta è uguale PC + [20-31] (i 12 bit dell'istruzione).

Esempio: se in memoria si è definito staticamente il

```
vettore: .word 0x10008000, 0x10008200, 0x10001088, 0x10008444
```

e al registro **t0** è assegnato il valore **3** allora

```
vj t0, vettore
```

salterà all'indirizzo **0x10008444** (che è l'elemento con indice 3 del vettore).

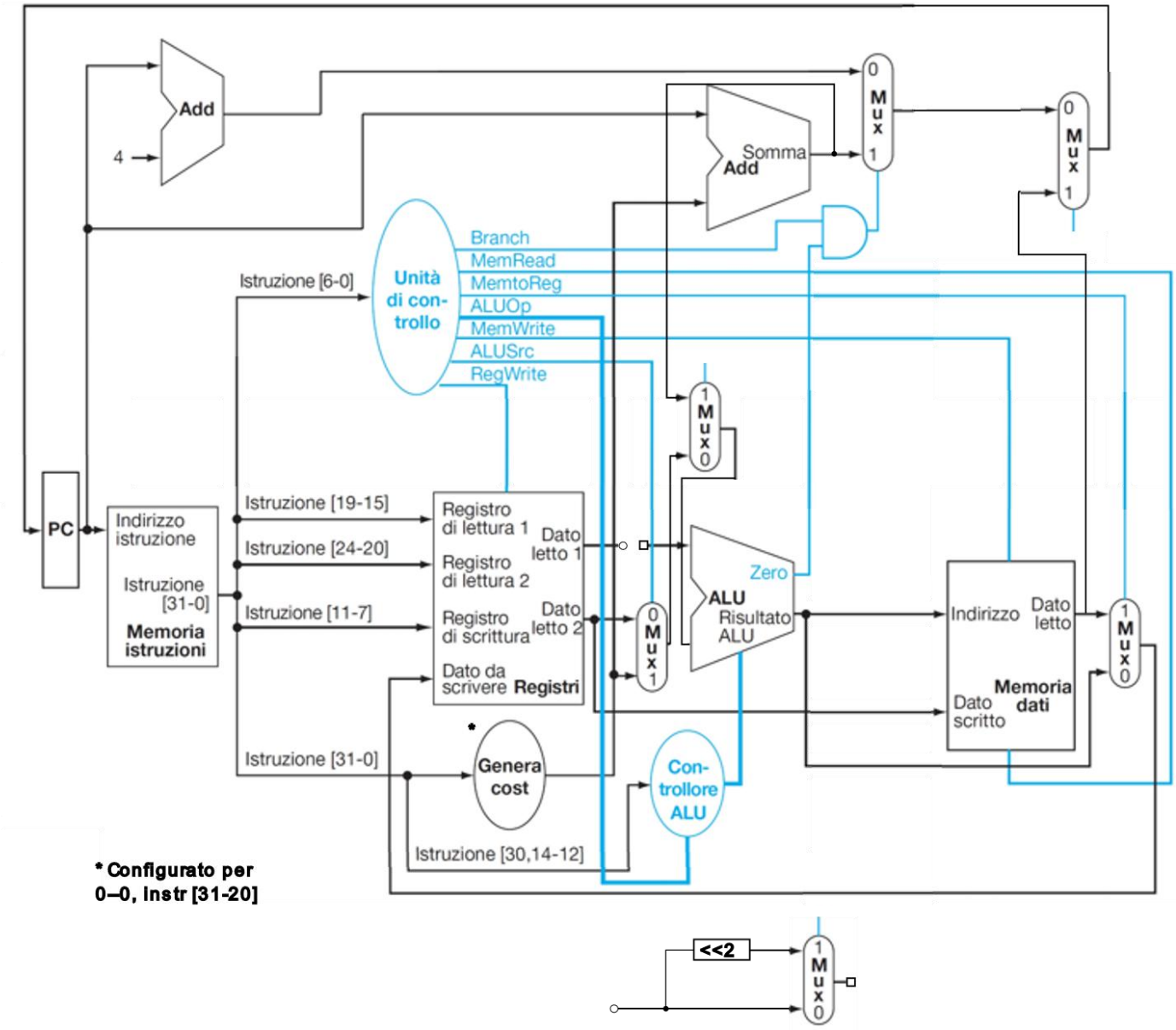
a) si disegnino le modifiche necessarie a realizzare la funzione, aggiungendo tutti gli eventuali MUX, segnali di controllo, bus, ALU e sommatori (ecc.) che ritenete necessari.

b) indicate i valori di tutti i segnali di controllo, in modo da eseguire l'istruzione **vj.**

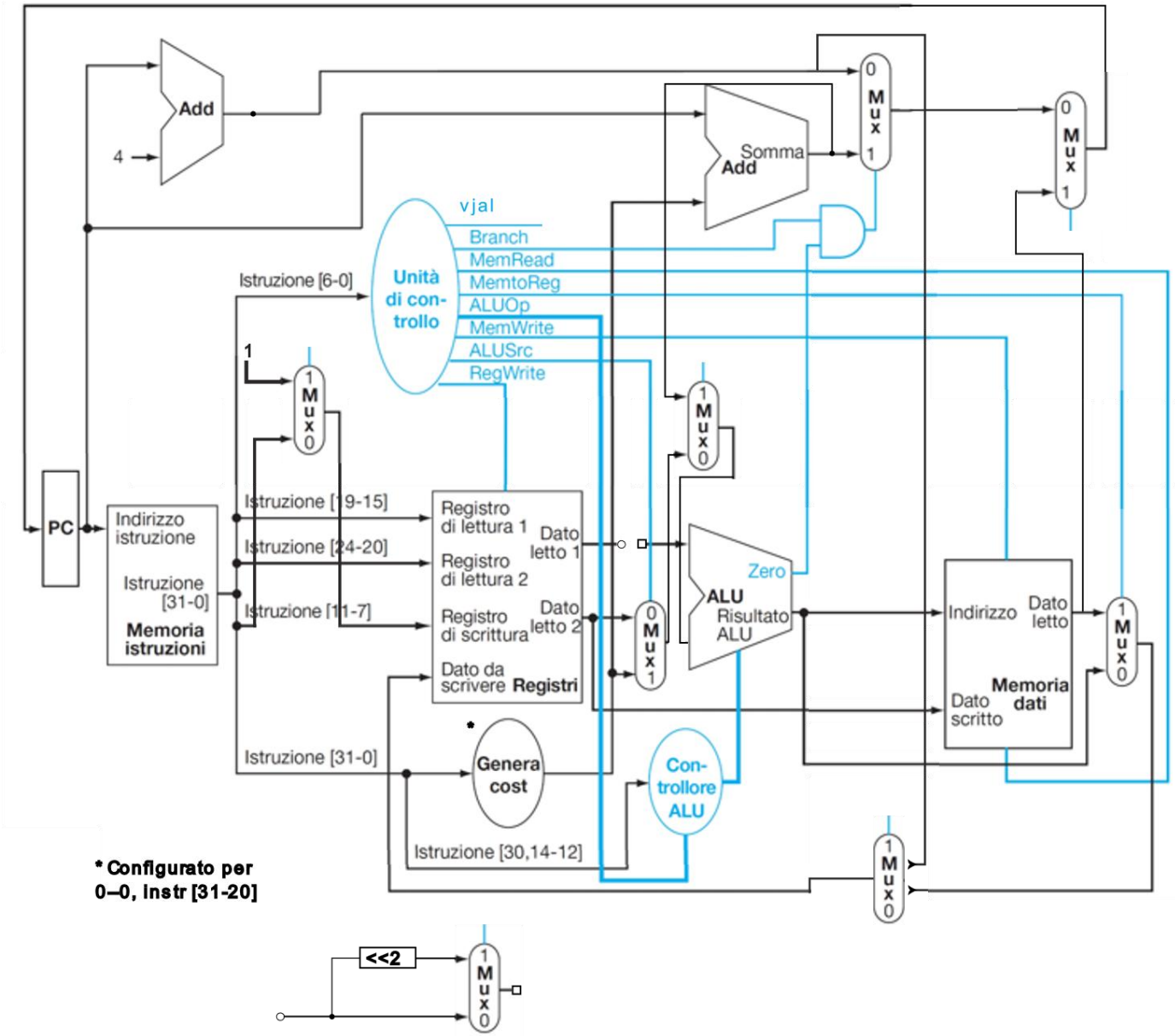
c) tenendo conto che il tempo di accesso ai registri (sia in lettura che scrittura) è di **25ns**, l'accesso alla memoria impiega **75ns**, la ALU e i sommatori impiegano **100ns** e ignorando gli altri ritardi, calcolate il tempo di esecuzione minimo della istruzione **vj** e indicate se è necessario aumentare il periodo di clock della CPU per poter svolgere questa nuova istruzione.

Esercizio per casa: modificare `vj` in **`vjal`** (salvando il `pc` corrente in `ra` prima del salto)

Esercizio per casa: da vj a vjal (soluzione /a)



Esercizio per casa: vjal (soluzione /a)



vjal rs, constant (soluzione /b, c)

(b) Segnali dalla Control Unit									
VJal	Reg Dest	ALU Src	Mem toReg	Reg Write	Mem Read	Mem Write	Branch	ALU Op1	ALU Op0
1	X	1	X	1	1	0	X	0	0

Accesso a memorie = 75ns, accesso ai registri = 25ns, ALU e sommatori = 100ns

(c) Tempo di esecuzione				
Fetch 75ns	Reg[rs] 25ns	add = rs<<2 + const. 100ns	jumpTo = Mem[add] 75ns	PC ← jumpTo 0ns
PC+4 100ns		Reg[31] =PC+4 25ns		
Totale: 275ns				

Non è necessario incrementare il periodo di clock

Istruzione	Instr. Fetch	Instr. Decode	Execution	MEM	Wr. Back	Totale
lw	75	25	100	75	25	300